GIT QUEST | 30-DAY GIT LEARNING SERIES

Day 4: git add and Commit Messages

Week 1: First Steps | Level: Absolute Beginner

Created by Rutuja Naik

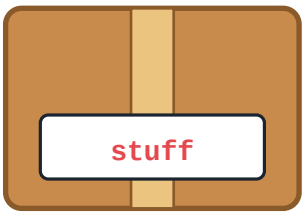
Today's mission

Yesterday you learned the 3 areas of Git. Today you learn to pack the box properly with `git add` and to write a **label** (commit message) your future self will understand.

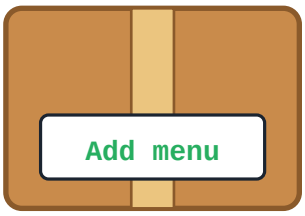
- Stage one file, several files, or everything with `git add`
- Write commit messages that are short, clear and useful
- Keep each commit to one logical change
- Read your history with `git log --oneline`

1. The big idea: the label on the box

When you seal a box you stick a label on it. A label like **stuff** tells nobody anything. A label like **Winter clothes, size M** saves you an hour next year. A commit message is the label on your save point.



Bad label: what is inside?



Good label: clear at a glance

> **The magic test**
> Finish this sentence: **"If applied, this commit will _____."** "...Add menu" makes sense. "...stuff" does not.

2. The git add toolbox

Habit: run `git status` before `git add .` so you know exactly what you are about to pack.

Command	What it stages	When to use it
git add menu.txt	One file	The safest habit
git add notes.txt todo.txt	Several named files	Files that belong together
git add *.md	Every file matching a pattern	All Markdown files, for example
git add .	Every change in this folder	After checking git status first

3. Writing good commit messages

The anatomy

```
feat: Add dessert section to menu
  |    '-- short summary: starts with a verb, no full stop
  '-- type (optional): feat, fix, docs, chore
```

5 golden rules

1. Keep the summary under **50 characters**
2. Start with a **verb**: Add, Fix, Update, Remove
3. **No full stop** at the end
4. **One commit = one logical change**
5. Say **what** changed (use a longer body for **why**)

Bad vs good

Bad	Good
stuff	Add menu with three dishes
fix	Fix typo in menu title
asdfgh	Add notes file
final changes!!!	Update todo list with deadlines
Added login, fixed bug, updated docs	Three separate commits, one per change

Commit types you will see everywhere

Type	Use it for
feat:	A new feature
fix:	A bug fix
docs:	Documentation only
chore:	Housekeeping and small upkeep
refactor:	Tidying up without changing what it does

4. Step by step

Step 1: Start a practice repo

```
$ git init -b main commit-lab
$ cd commit-lab
```

Step 2: Create three files

```
$ echo "Pasta" > menu.txt
$ echo "Ideas" > notes.txt
$ echo "Call mom" > todo.txt
```

Step 3: Add ONE file and commit

```
$ git add menu.txt
$ git commit -m "Add menu"
[main (root-commit) c92f6e1] Add menu
 1 file changed, 1 insertion(+)
```

Step 4: Add TWO files at once

```
$ git add notes.txt todo.txt
$ git commit -m "Add notes and todo files"
[main a17d803] Add notes and todo files
 2 files changed, 2 insertions(+)
```

Step 5: Edit, check, add all, commit with a body

```
$ echo "Salad" >> menu.txt
$ echo "Buy milk" >> todo.txt
$ git status
$ git add .
$ git commit -m "Update menu and todo lists" -m "Added salad and milk reminder."
```

Step 6: Read your history

```
$ git log --oneline
5e9b2c4 (HEAD -> main) Update menu and todo lists
a17d803 Add notes and todo files
c92f6e1 Add menu
```

5. Cheat sheet

Command	What it does
git add <file>	Stage one file
git add a.txt b.txt	Stage several files
git add .	Stage all changes in this folder
git commit -m "message"	Save with a short label
git commit -m "s" -m "body"	Save with a label and a longer note
git log --oneline	Show your history, one line per commit

6. Today's 3 commits

#	File	Content	Commit message
1	day04-commit-messages/lesson.md	Today's lesson with rules and examples	docs(day04): add lesson on git add and commit messages
2	day04-commit-messages/demo.tape	VHS script that records the terminal GIF	feat(day04): add VHS tape for commit message demo
3	exercise.md and days-1-to-4-summary.md	Challenge, plus a recap of Days 1 to 4	docs(day04): add exercise and days 1-4 summary

7. Mermaid diagram GitHub draws for you

````
```mermaid
gitGraph
  commit id: "Add menu"
  commit id: "Add notes and todo files"
  commit id: "Update menu and todo lists"
```
````

8. Challenge and quiz

Challenge: rewrite these labels so each one passes the magic test: `stuff`, `fix`, `final changes!!!`

Q1. Which is better: "Added menu" or "Add menu"?

Q2. What does git add . do?

Q3. How many changes should one commit contain?

Answers: 1) Add menu. 2) Stages all changes in this folder. 3) One logical change.

> **Coming up: Day 5**
> **git log, your time machine.** Travel back through your history and see every save point.